

TypeScript Practice

Day 1 Lab

Objective: TypeScript Functions, Interfaces, and Types

Task & Build Specifications: This lab includes four parts to give you lots of TypeScript practice. Complete the entire lab in one project using one .ts file.

Write all functions as arrow functions (e.g. `const myFunction = (param: string): void => { ... }`), since that's the style you'll see throughout the Playwright portion of this workshop.

Tallest Mountain

- Declare an interface called `Mountain` that contains the following properties:
 - `name` - string
 - `height` - number
- Declare an array called `mountains` which is an array of type `Mountain`.
- Fill the array with the following mountains:

name	height
Kilimanjaro	19341
Everest	29029
Denali	20310

- Declare a function called `findNameOfTallestMountain`. It takes one parameter, an array of `Mountain` objects. It returns a string, the name of the tallest mountain in the given array.
- Call `findNameOfTallestMountain`, passing it your `mountains` array as an argument.
- Store the result of the function call (the return value) in a variable and then `console.log` the variable. (Hint: It will print "Everest".)

Products

- Declare an interface called `Product` that contains the following properties:
 - `name` - string
 - `price` - number
- Declare an array called `products` which is an array of type `Product`.
- Fill the array with a few products of your own choosing.
- Declare a function called `calcAverageProductPrice`. It takes one parameter, an array of `Product` objects. It returns a number, the average price of all the products provided as an argument.
- Call `calcAverageProductPrice`, passing it your `products` array as an argument.
- Store the result of the function call (the return value) in a variable and then `console.log` the variable.

Inventory

- Declare an interface called `InventoryItem` that contains the following properties:
 - `product` - `Product` (from above)
 - `quantity` - number
- Declare an array called `inventory` which is an array of type `InventoryItem`.
- Fill the array with the following information.

product.name	product.price	quantity
motor	10.00	10
sensor	12.50	4
LED	1.00	20

- Declare a function called `calcInventoryValue`. It takes one parameter, an array of `InventoryItem` objects. It returns a number, the total value of all the products in the inventory array provided as an argument.
- This is calculated as follows: For each `InventoryItem` take the product price times the quantity. Add these together for all the items. For the above data, the total will be $10.00 \times 10 + 12.5 \times 4 + 1.00 \times 20 = 170$.
- Call `calcInventoryValue`, passing it your `inventory` array as an argument.
- Store the result of the function call (the return value) in a variable and then `console.log` the variable. (Hint: It prints 170.)

Student Grades

- Declare an interface called `Student` that contains the following properties:
 - `name` - string
 - `grade` - this should be a union type: a number, or the string "incomplete"
- Declare an array called `students` which is an array of type `Student`.
- Fill the array with the following students:

name	grade
Amir	92
Priya	78
Jordan	"incomplete"
Sam	"incomplete"

- Declare a function called `printStudentReport`. It takes one parameter, an array of `Student` objects. It does *not* return a value (its return type is `void`).
- For each student in the array, `console.log` a line with their name and grade:
 - If the grade is a number, print: "Amir: 92"
 - If the grade is "incomplete", print: "Jordan: incomplete"

Requirements

- Mountain interface exists with name (string) and height (number) properties.
- mountains array exists with given data.
- Product interface exists with name (string) and price (number) properties.
- products array exists with several objects of data.
- InventoryItem interface exists with product (Product) and quantity (number) properties.
- inventory array exists with given data.
- findNameOfTallestMountain takes Mountain array parameter and returns correct string.
- calcAverageProductPrice takes Product array parameter and returns correct number.
- calcInventoryValue takes InventoryItem array parameter and returns correct number.
- Student interface exists with name (string) and grade (number | "incomplete") properties.
- students array exists with given data.
- printStudentReport takes a Student array parameter, returns void, and correctly distinguishes numeric grades from "incomplete".
- All functions are declared as arrow functions.
- All of the functions created are called correctly and results are stored and logged (where applicable).